

This is APIO, not APOI

Alice has an integer x and wants to send it to Bob. To do this, she sends Bob a string that only consists of characters I or O . However, Chris, an APIO lover, doesn't like the substring OI . After Alice sends the strings, he intercepts the string, performs the following operations **at most** k times, and sends the modified string to Bob:

- Choose an OI substring and replace it with IO .

For example, if Alice sends OOIOI to Bob and $k = 1$, there are three candidate strings that Bob may receive: OOIOI , OIOOI , OOIIO .

Finally, Bob receives a string that consists of characters I or O and needs to recover the original integer x from Alice. Please help Alice and Bob design a strategy to do so. Your score will depend on the maximum length of the string that Alice sends.

Implementation Details

You need to implement two procedures; one for Alice, and another one for Bob.

The procedure you should implement for **Alice** is:

```
std::string Alice(int x, int k)
```

- x : the integer Alice wants to send to Bob.
- k : the maximum number of operations Chris performs.
- This procedure is called at most 10^5 times in each test case.

This procedure should return a string that only consists of characters I or O . The length of string should be at most 120.

The procedure you should implement for **Bob** is:

```
int Bob(std::string s, int k)
```

- s : a string that was sent by Alice and modified by Chris.
- k : the maximum number of operations Chris performs.

- This procedure may be called multiple times in each test case, **exactly once** for each corresponding `Alice` call. The order of `Bob` procedure calls is not necessarily the same as the order of the corresponding `Alice` calls.

This procedure should return an integer x , the integer that Alice sends.

In the actual grading, a program that calls the above procedures is run **exactly two times**.

- During the first run of the program:
 - `Alice` is called at most 10^5 times.
 - The returned values are stored by the grading system. Then, for each string, the grading system generates a new string using the replacing operations described in the statement.
- During the second run of the program:
 - `Bob` is called multiple times, with the strings generated by the grading system in the first run as the argument s . Note that Bob also receives k .

Constraints

- $0 \leq x \leq 2^{30} - 1$
- $1 \leq k \leq 3$

Subtasks and Scoring

In any of the test cases, if at least one of the following conditions hold, then the score of your solution for that test case will be 0 (reported as `Output isn't correct` in CMS):

- The string Alice returns consists of character other than `I` and `O`.
- The maximum length of the string that Alice sends is larger than 120.
- The return value of any call to procedure `Bob` is incorrect.

Otherwise, let L be the maximum length of the string that Alice sends. Define score ratio S as follows:

Condition	S
$60 < L \leq 120$	$\frac{126-L}{150}$
$46 < L \leq 60$	$\frac{71-L}{25}$
$L \leq 46$	1

Then, the score is calculated as follows:

Subtask	Score	Additional Constraints
1	$25 \cdot S$	$k = 1$
2	$75 \cdot S$	No additional constraints.

Example

Let $k = 3$. Consider the following sequence of calls during the first run of the program:

Procedure call	Return value
Alice(4, 3)	"OI O I"
Alice(6, 3)	"OI O IO O I"

After that, the grading system performs the replacing operations. Now, the first string becomes `IIOO` and the second becomes `OIOIOOI`. Then, in the second run of the program, the following calls are made:

```
Bob("IIOO", 3)
Bob("OIOIOOI", 3)
```

Bob should return 4 in the first call and 6 in the second call.

Sample Grader

The sample grader calls both `Alice` and `Bob` in the same run, which is different from the actual grader.

Input format:

```
n k
X[0] X[1] X[2] ... X[n-1]
```

The sample grader will call `Alice` n times, and $X[i]$ will be used as the argument of the $(i + 1)$ -th call.

Output format:

```
Wrong Answer: Reason
```

or

Accepted: L

Here, L is the maximum length of the string that Alice sends. Note that you can use the sample grader with any value of n and k .